

LAB5 - REPORT

Martí Gallego Giménez u251445
Mateu Soria Waha u251380

1. Introducció

Aquest Lab implementa la regressió lineal amb Gradient Descent (GD) i Stochastic Gradient Descent (SGD) en C++. L'objectiu principal és traduir la implementació de Java del Lab 4 a C++, aplicant els conceptes d'herència, classes abstractes i polimorfisme en aquest nou llenguatge.

Les classes Vector, Record, Dataset i Model ja estaven implementades i proporcionades en el codi base. La nostra tasca ha consistit en implementar les quatre classes restants:

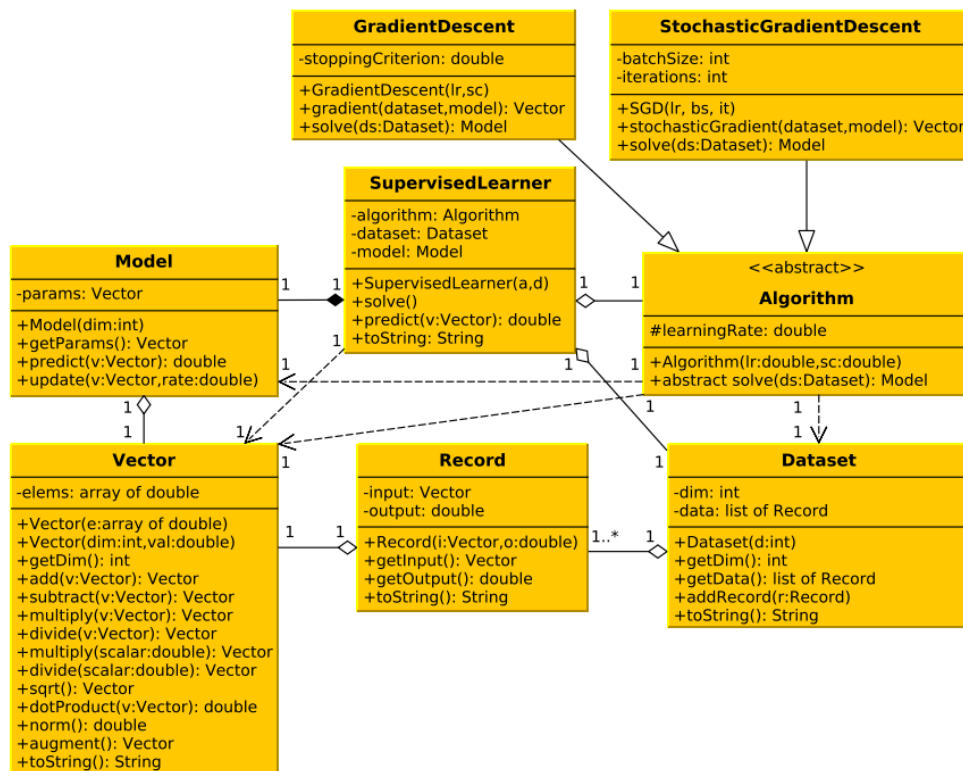
- **Algorithm:** Classe abstracta que defineix l'estructura base per a tots els algoritmes d'aprenentatge.
- **GradientDescent:** Implementació del descens de gradient complet (batch gradient descent).
- **StochasticGradientDescent:** Implementació del descens de gradient estocàstic amb minibatches.
- **SupervisedLearner:** Classe que gestiona l'entrenament i la predicció dels models.

A diferència del Lab 4 en Java, aquesta implementació en C++ no inclou datasets estandarditzats ni les seves transformacions, simplificant així el codi.

2. Descripció de la solució

El Lab s'ha implementat dins de la carpeta anomenada Lab5 dins del nostre repositori de Github. Conté els arxius de les classes proporcionades (Vector.h, Record.h, Dataset.h i Model.h) i els arxius implementats per nosaltres (Algorithm.h, GradientDescent.h, StochasticGradientDescent.h, SupervisedLearner.h i TestLearner.cpp). Els resultats de les proves es mostren per terminal.

Les relacions entre les diferents classes són les següents:



2.1. Classe Algorithm

Aquesta classe abstracta proporciona l'estructura base per a tots els algorismes d'aprenentatge supervisat.

Atributs:

- **double learningRate**: Taxa d'aprenentatge (α) protegida que controla la mida dels passos durant l'optimització.

Constructor:

- **Algorithm(double lr)**: Inicialitza la taxa d'aprenentatge amb el valor proporcionat. Utilitza la llista d'inicialització de C++ per assignar el valor abans que s'executi el cos del constructor.

Mètode abstracte:

- **virtual Vector solve(const Dataset & data, Model & model) = 0**: Mètode virtual que ha de ser implementat per cada subclasse. La sintaxi = 0 el defineix com a mètode abstracte.

2.2. Classe GradientDescent

Implementa el descens de gradient complet (batch gradient descent), que utilitza tot el dataset en cada iteració per calcular el gradient. Hereta de la classe Algorithm.

Atributs:

- **int maxIterations:** Nombre màxim d'iteracions que l'algoritme pot realitzar. Aquest atribut és privat ja que només s'utilitza internament en aquesta classe.

Constructor:

- **GradientDescent(double lr, int maxIter):** Rep la taxa d'aprenentatge i el nombre màxim d'iteracions. Utilitza la llista d'inicialització per cridar primer el constructor de la classe base Algorithm(lr) i després inicialitzar maxIterations(maxIter).

Mètodes:

- **Vector solve(const Dataset & data, Model & model) override:** Implementa el bucle principal del descens de gradient.

El procés és el següent:

1. Obté tots els registres del dataset amb data.getData().
2. Itera fins a maxIterations vegades.
3. A cada iteració, inicialitza un vector gradient a zero amb dimensió $dim + 1$ (per incloure el bias).
4. Per cada registre del dataset:
 - Augmenta el vector d'entrada afegint 1 al final (bias) amb augment().
 - Calcula la predicció actual amb model.predict(input).
 - Calcula l'error com a prediction - output.
 - Calcula el gradient parcial multiplicant l'input per l'error.
 - Acumula aquest gradient parcial al gradient total.
5. Calcula el gradient mitjà dividint pel nombre de registres n.
6. Actualitza el model cridant model.update(gradient, learningRate).
7. Retorna un vector (en aquesta implementació, retorna un vector amb un sol element 0, però podria retornar informació sobre les losses).

2.3. Classe StochasticGradientDescent

Implementa el descens de gradient estocàstic (SGD), que utilitza minibatches aleatoris en cada iteració per actualitzar el model.

Atributs:

- **int maxIterations:** Nombre màxim d'iteracions (èpoques) que l'algoritme realitzarà.
- **int batchSize:** Mida del minibatch.

Constructor:

- **StochasticGradientDescent(double lr, int maxIter, int batch):** Rep la taxa d'aprenentatge, el nombre d'iteracions i la mida del batch. Crida el constructor de la classe pare i inicialitza els dos atributs privats.

Mètodes:

- **Vector solve(const Dataset & data, Model & model) override:** Implementa el bucle principal del SGD. El procés és el següent:
 1. Obté tots els registres del dataset.
 2. Inicialitza un generador de nombres aleatoris utilitzant *std::random_device* i *std::mt19937* de la llibreria *<random>* de C++.
 3. Per cada iteració:
 - Crea un vector d'índexs amb tots els índexs dels registres.
 - Barreja aquests índexs aleatòriament amb *std::shuffle()*.
 - Determina la mida real del batch com el mínim entre *batchSize* i el nombre total de registres *n*.
 - Crea el minibatch seleccionant els primers *actualBatchSize* registres segons els índexs barrejats.
 - Inicialitza un vector gradient a zero.
 - Per cada registre del batch:
 - Augmenta el vector d'entrada.
 - Calcula la predicció i l'error.
 - Calcula el gradient parcial.
 - Acumula el gradient.
 - Calcula el gradient mitjà del batch dividint per la mida del batch.
 - Actualitza el model.
 4. Retorna un vector (similar a *GradientDescent*).

2.4. Classe SupervisedLearner

Aquesta classe gestiona tot el procés d'aprenentatge supervisat, encapsulant l'algoritme i el model. És la interfície principal per entrenar models i fer prediccions.

Atributs:

- **Algorithm * algorithm:** Punter a l'algoritme d'aprenentatge que s'utilitzarà per entrenar el model.
- **Model * model:** Punter al model que serà entrenat.

Constructor:

- **SupervisedLearner(Algorithm * algo, int inputDim):** Rep un punter a l'algoritme i la dimensió dels vectors d'entrada. Inicialitza l'atribut *algorithm* amb el punter rebut i crea un nou Model amb dimensió *inputDim + 1* (per incloure el bias). El model s'emmagatzema com a punter perquè es crea amb *new*.

Destructor:

- **~SupervisedLearner():** Aquest és el destructor de la classe, indicat pel símbol ~ abans del nom de la classe. El destructor s'executa automàticament quan un objecte SupervisedLearner és destruït, i nosaltres l'utilitzem per alliberar la memòria del model.

Mètodes:

- **void train(const Dataset & data):** Entrena el model cridant el mètode solve() de l'algoritme.
- **double predict(const Vector & input) const:** Rep un vector d'entrada (sense augmentar), l'augmenta afegint el bias, i retorna la predicció del model.
- **double evaluate(const Dataset & data) const:** Calcula l'error quadràtic mitjà (MSE) del model sobre un dataset. Per cada registre, calcula la predicció, l'error, l'eleva al quadrat i l'acumula. Finalment divideix pel nombre de registres per obtenir la mitjana. Aquest mètode és útil per avaluar la qualitat del model entrenat.
- **friend std::ostream & operator<<(std::ostream & os, SupervisedLearner & sl):** El modificador friend indica que aquesta funció és externa a la classe però té accés als seus membres privats. Simplement imprimeix el model desreferenciant el punter.

2.5. Classes proporcionades

Tot i que no les hem implementat nosaltres, és important entendre breument les classes que ens van proporcionar:

Vector: Representa un vector matemàtic amb operacions bàsiques com suma, resta, multiplicació, divisió, producte escalar, norma i augmentació (afegir 1 al final per al bias). Utilitza internament `std::vector<double>` per emmagatzemar els elements.

Record: Representa un parell d'entrenament (input, output). Emmagatzema un Vector com a entrada i un double com a sortida.

Dataset: Conté una llista de Records. Proporciona mètodes per obtenir la dimensió, obtenir totes les dades i afegir nous registres. Utilitza `std::vector<Record>` per emmagatzemar els registres.

Model: Representa el model de regressió lineal amb els seus paràmetres (pesos). Proporciona el mètode predict() per fer prediccions (calculant el producte escalar entre els paràmetres i l'input augmentat) i update() per actualitzar els paràmetres durant l'entrenament.

2.6. TestLearner.cpp

Conté el mètode main() que prova totes les funcionalitats implementades. El programa realitza diverses proves:

Test de Vectors: Prova les operacions bàsiques amb vectors per verificar que la classe Vector funciona correctament. Crea vectors, els suma, resta, multiplica, divideix, calcula productes escalars i normes.

Creació del dataset d'entrenament: Crea un Dataset de dimensió 2 amb 5 registres que segueixen aproximadament la relació $y = 2x_1 + 3x_2 + 1$. Els registres són:

- (1.0, 2.0) → 9.0
- (2.0, 1.0) → 8.0
- (3.0, 3.0) → 16.0
- (0.0, 1.0) → 4.0
- (4.0, 2.0) → 15.0

Test amb Gradient Descent:

- Crea un objecte GradientDescent amb learning rate 0.01 i 1000 iteracions.
- Crea un SupervisedLearner amb aquest algoritme.
- Mostra el model inicial (amb paràmetres inicialitzats a 1).
- Entrena el model cridant train().
- Mostra el model entrenat amb els nous paràmetres.
- Calcula i mostra l'error MSE sobre el dataset d'entrenament.
- Fa una predicció per un vector de test (1.5, 2.5).

Test amb Stochastic Gradient Descent:

- Crea un objecte StochasticGradientDescent amb learning rate 0.01, 2000 iteracions i batch size 2.
- Segueix el mateix procediment que amb GradientDescent.
- Compara els resultats amb els obtinguts per GD.

Els resultats mostren que ambdós algoritmes convergeixen a solucions similars, encara que SGD pot tenir lleugerament més variabilitat degut a la naturalesa estocàstica del seu procés d'optimització.

3. Conclusions

La implementació del Lab 5 s'ha completat amb èxit, traslladant els conceptes de classes abstractes, herència i polimorfisme de Java a C++.

Aprenentatge sobre C++: Aquest lab ha estat fonamental per entendre les diferències entre Java i C++. La gestió manual de memòria amb new i delete, l'ús de referències constants per optimitzar el pas de paràmetres, les directives de preprocessador per evitar inclusions duplicades, i la sintaxi diferent per a herència i mètodes abstractes són conceptes clau que hem après.

Polimorfisme en C++: Hem comprovat que el polimorfisme funciona de manera similar a Java però amb algunes particularitats. L'ús de punters a la classe base (Algorithm*) permet treballar amb qualsevol subclasse (GradientDescent o StochasticGradientDescent) de forma

transparent. Els mètodes virtuals i l'especificador override garanteixen que les crides polimòrfiques funcionin correctament.

Simplicitat vs funcionalitat: A diferència del Lab 4, aquesta implementació no inclou datasets estandarditzats ni transformacions. Això ha simplificat considerablement el codi, permetent-nos centrar-nos en els conceptes fonamentals d'herència i polimorfisme en C++. Tot i aquesta simplificació, els algorismes implementats funcionen correctament i produeixen resultats similars als obtinguts en Java.

En conclusió, aquest lab ha estat una excel·lent introducció a la programació orientada a objectes en C++, mostrant com els mateixos conceptes de POO s'apliquen de forma diferent segons el llenguatge utilitzat.